

# Power Routing for Reconfigurable Modular Robots

Concise code audit, strict flowcharts, related work and minimal tests

CONCISE EDITION | 20 JULY 2026

## Bottom line

Dual-mode is a deterministic nearest-source router. Balanced mode is a greedy, count-based post-processor that moves modules only between equal-distance batteries. Neither method enforces electrical capacity, battery isolation, radially, voltage, current, state of charge, or safe actuation.

### DUAL-MODE

Exact unweighted shortest paths; weighted routing follows Dijkstra with a tolerance-based owner tie rule.

### BALANCED

Strictly improves a lexicographic count objective until no improving single equal-distance move remains.

### DEPLOYMENT

Do not energize the union of balanced paths without source-isolation, cycle, and hardware-limit checks.

## REPORT MAP

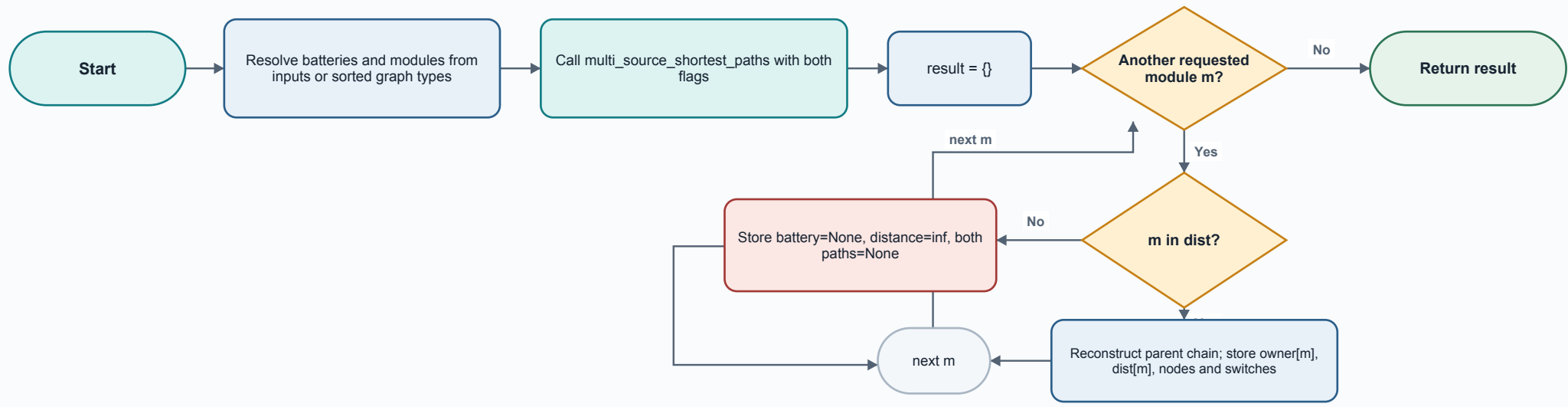
Five content pages: findings, theory, literature, experiments, recommendation. Four separate flowchart sheets: dual API, BFS, Dijkstra, balanced.

**Recommended direction**    **Safety-constrained, capacity-aware lifetime optimization**

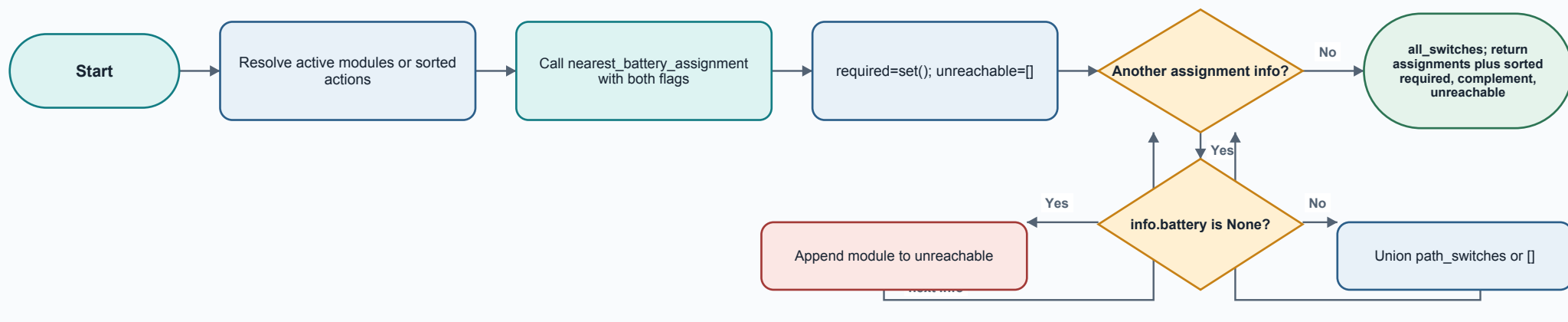
# Dual-mode: assignment and switch-plan APIs

Literal control flow from power\_routing\_duo\_mode.py. 'Enabled' below corresponds to the code's open\_ = True state.

## A. nearest\_battery\_assignment | default respect\_switch\_state = True



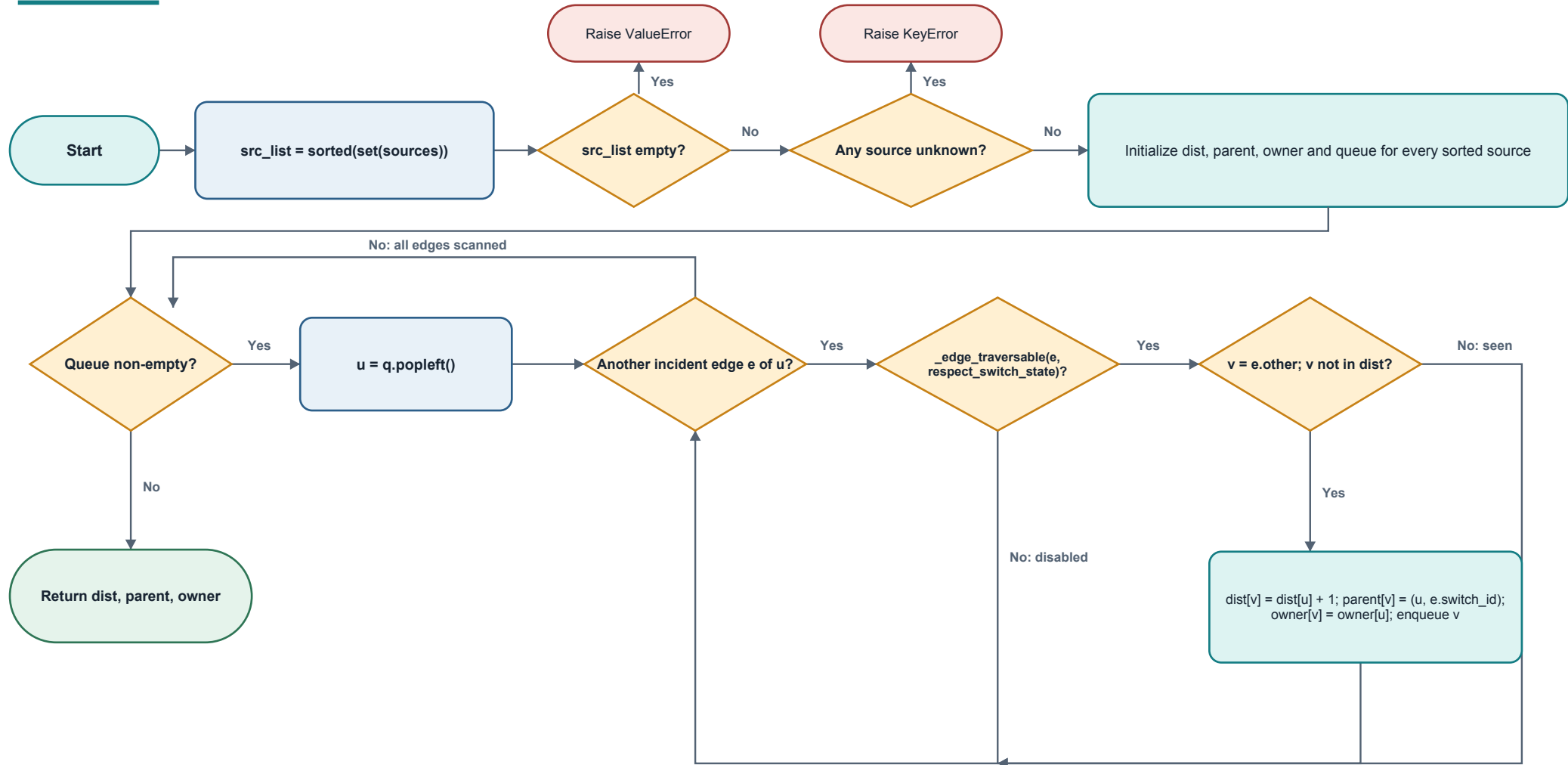
## B. recommend\_switch\_plan | default respect\_switch\_state = False (physical-topology planning)



API boundary: returns a recommendation dictionary; no switch is actuated or electrically validated.

# Unweighted engine: multi-source BFS

Every traversed enabled switch costs one hop. First discovery fixes distance, parent and owner.

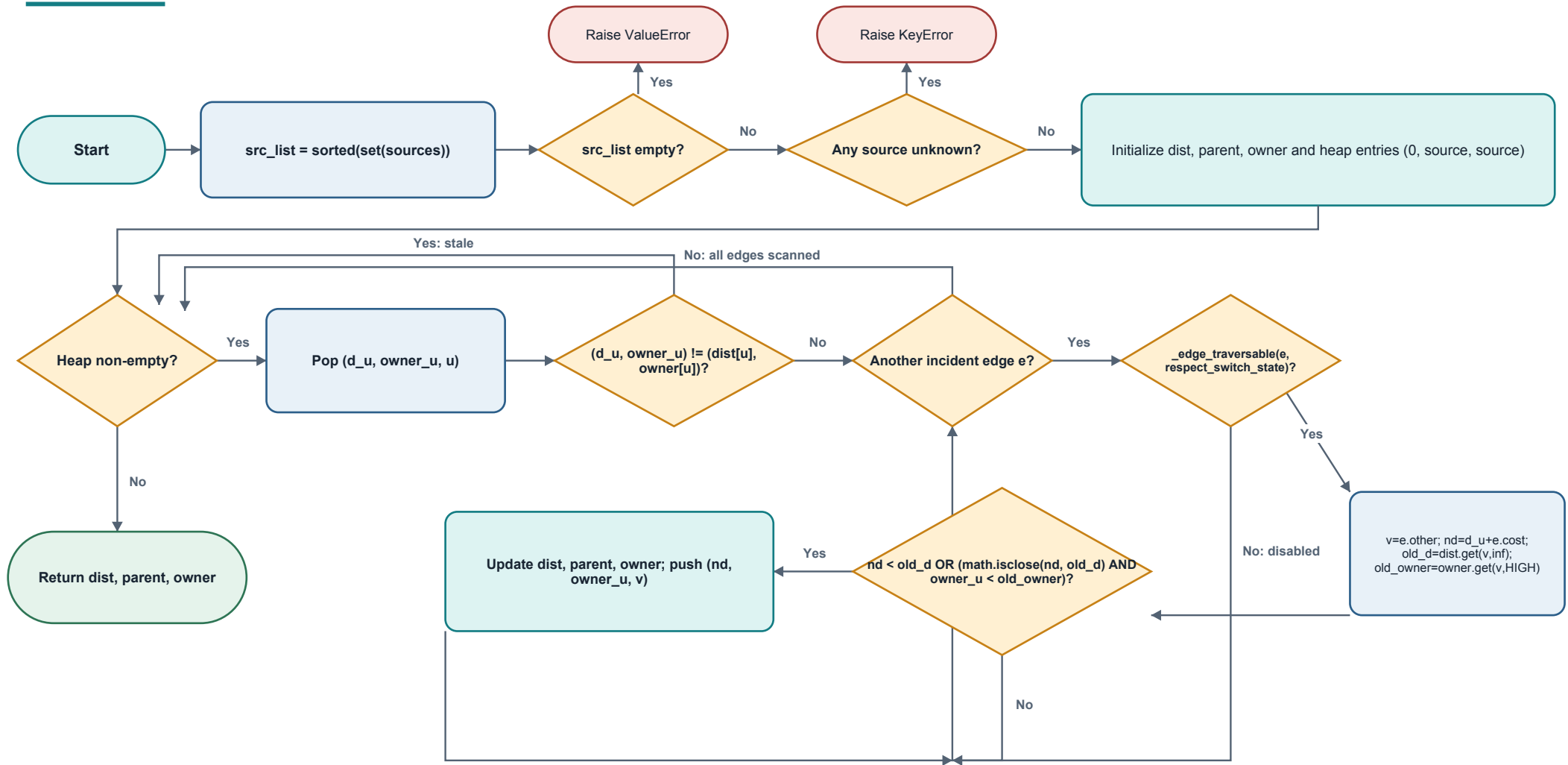


Traversable iff (not respect\_switch\_state) or switch\_open.get(e.switch\_id, False); adjacency lists use insertion order.

Skip branches return to the same edge loop. The only normal exit is an empty queue; there is no equal-distance relaxation.

# Weighted engine: multi-source Dijkstra

Heap entries are (distance, owner\_id, node). Edge costs are nonnegative; equality uses `math.isclose`.

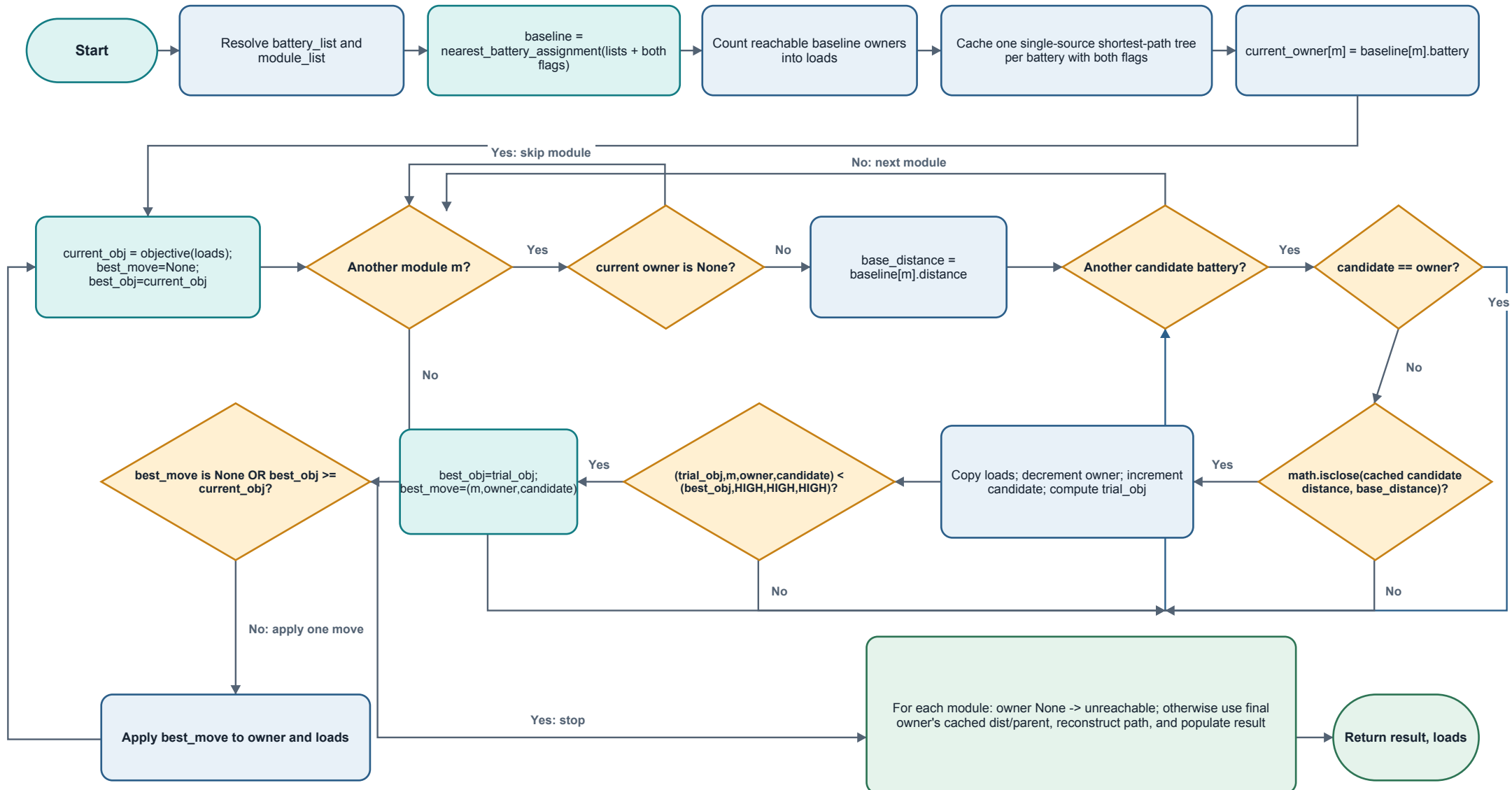


**Strict caveat: sources are not immutable. A zero-cost route from a smaller-ID source can reparent another source.**

Also, `math.isclose` is tolerance equality; a slightly longer path can win the owner tie within that tolerance.

# Balanced mode: greedy equal-distance rebalancing

Literal flow of rebalanced\_nearest\_battery\_assignment. It returns assignments and count loads, not a switch plan.



Tie behavior is literal: rebuilding the HIGH sentinel lets later equal-objective moves overwrite earlier ones; input order can change the move.

# What the code guarantees - and what it does not

Theoretical comparison of the two current repository methods.

## ALGORITHM COMPARISON

| Method    | Optimization logic                                                                                                           | Guarantee                                                                                                                                         | Main omission                                                                                             |
|-----------|------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| Dual-mode | One multi-source BFS (hop cost) or Dijkstra (nonnegative additive edge cost). Sorted source IDs break ownership ties.        | Unweighted: exact nearest source and path. Weighted: nonnegative-cost Dijkstra logic, but math.isclose can accept a near-equal longer owner path. | Independent paths only; no shared capacity, electrical feasibility, lifetime, or actuation safety.        |
| Balanced  | Start from dual; cache one tree per battery; move one module only to an equal-distance battery when the load tuple improves. | Finite strict descent to a one-move local optimum for (spread, SSE, descending load tuple). Route distance is preserved within math.isclose.      | Counts modules, not power. No global optimality, source isolation, radially, or balanced switch-plan API. |

## CRITICAL IMPLEMENTATION FACTS

- Balanced recommend\_switch\_plan still calls nearest\_battery\_assignment; it does not expose the rebalanced result.
- Unioning final balanced paths can connect multiple batteries in one component or create a cycle; no check rejects either case.
- The best-move sentinel is rebuilt each comparison. Equal-objective moves overwrite earlier choices, so module/battery input order can affect output.
- Zero-cost weighted edges can reparent a source. Example: B1--0--X--0--B2--1--M lets B1 own B2 and produces a path through both batteries.
- Neither method models current, voltage, source/link capacity, conversion loss, battery state of charge, task demand, or switch transition cost.

## INTERPRETATION

**Use dual-mode as a routing baseline, not as an electrical controller.**

Balanced is useful only when module count is a credible load proxy and every chosen battery-to-module path is independently safe. In hardware, topology feasibility and protection limits must be hard constraints that run before any optimization preference.

# Related power-routing optimization methods

Five closest direct or transferable baselines; none is a drop-in replacement for this repository.

| Method                        | Objective and constraints                                                                                                               | Evidence                      | Comparison with this repository                                                                           |
|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|-------------------------------|-----------------------------------------------------------------------------------------------------------|
| Campbell et al. 2005 [1]      | Distributed construction of supply and ground rails using local gradient, randomized, negotiation, or lattice rules; tolerates defects. | SPICE simulation              | Closest direct routing work. Stronger two-rail topology model; weaker global route-cost optimality.       |
| Chen et al. 2016 [2]          | Local consensus and donor/receiver decisions target maximum robot operation time from remaining energy and consumption.                 | Numerical and ReMod3D         | Better lifetime objective than count balance; does not synthesize an isolated minimum-loss switch forest. |
| Chen et al. 2017 [3]          | Centralized and neighborhood allocation for maximum first-depletion time with source output-current limits.                             | Graphs to 1,274 nodes; ReBots | Best direct capacity-aware allocation baseline; path loss and actual route selection are omitted.         |
| Liang et al. FreeBOT 2022 [4] | Incidence-matrix processing removes open, short, and cyclic invalid sharing components.                                                 | FreeBOT prototype             | Best topology-safety complement; does not optimize route cost or lifetime.                                |
| Wang et al. e-LAN 2017 [5]    | Capacity screening plus loss-aware lowest-cost source/path selection, including multi-source supply for large loads.                    | Case studies and simulation   | Closest transferable graph optimizer; electrical-router assumptions need adaptation to modular robots.    |

## Short synthesis

The closest combined reference design is: Chen 2017 for capacity-aware allocation, FreeBOT for topology rejection, e-LAN for physical loss/capacity costs, and Chen 2016 for the lifetime objective. Campbell 2005 is the distributed alternative when global topology is unavailable.

# Smallest useful experimental comparison

Seeded software simulations only. The repository algorithms were not modified; these are not hardware-safety tests.

Protocol

200 seeded 6x6 routing trials per outage rate; 200 seeded 3x3 energy-horizon trials. The exact oracles use restricted repository abstractions.

ROUTING AND TOPOLOGY

N = nearest, B = balanced; rates are percentages of trials

| Outages | Load spread N -> B | Route distance N / B | B: multi-battery component | B: cycle | B: global count-oracle match |
|---------|--------------------|----------------------|----------------------------|----------|------------------------------|
| 0%      | 10.745 -> 6.465    | 68.105 / 68.105      | 38.0%                      | 14.5%    | 97.0%                        |
| 10%     | 11.535 -> 7.215    | 73.355 / 73.355      | 38.0%                      | 10.0%    | 96.5%                        |
| 25%     | 12.450 -> 9.465    | 81.310 / 81.310      | 26.0%                      | 5.0%     | 99.5%                        |

FEASIBLE OPERATING HORIZON

Restricted capacity/demand model; mean feasible rounds

| Method                  | Mean rounds | Gap to exact | Exact match |
|-------------------------|-------------|--------------|-------------|
| Dual nearest, static    | 2.790       | 1.265        | 18.5%       |
| Balanced, static        | 3.120       | 0.935        | 29.0%       |
| Greedy lifetime-aware   | 3.960       | 0.095        | 91.0%       |
| Restricted exact oracle | 4.055       | 0            | 100.0%      |

READOUT

- Balanced reduced count spread by 24-40% while preserving route distance, as its equal-distance rule predicts.
- That count benefit did not make the independently unioned paths safe: source joining occurred in 26-38% and cycles in 5-14.5% of trials.
- A lifetime-aware greedy allocator was far closer to the restricted exact horizon than either static method.



# Recommended optimization architecture

Treat safety and hardware feasibility as constraints; optimize lifetime and loss only inside that feasible set.

## LEXICOGRAPHIC PRIORITY

### FEASIBILITY

1

Reach active modules; enforce source isolation or approved sharing, radiality, current, voltage, link/source capacity, and interlocks.

### LIFETIME

2

Maximize first-depletion time or minimize the worst normalized battery depletion using state of charge and task demand.

### EFFICIENCY

3

Minimize physically calibrated conversion and link loss after feasibility and lifetime are protected.

### SWITCHING

4

Minimize switch transitions and add hysteresis so small estimate changes do not chatter the topology.

## IMMEDIATE REPOSITORY ACTIONS

- Do not expose a balanced switch plan until an isolation/cycle/hardware-limit gate accepts the selected topology.
- Persist and compare the full best-move key so equal-objective selection is deterministic; add order-permutation tests.
- Either require strictly positive weighted edges or prevent source parent/owner updates; add the zero-cost regression case.
- Validate the final optimizer in hardware-in-the-loop before any live battery-sharing experiment.

## SELECTED REFERENCES

- [1] Campbell, Pillai and Goldstein, 'The Robot is the Tether,' IROS, 2005.
- [2] Chen, Collins and Shen, 'A Near-Optimal Dynamic Power Sharing Scheme for Self-Reconfigurable Modular Robots,' ICRA, 2016.
- [3] Chen, Collins and Shen, 'Maximal Operation Time Estimation ... with Output Current Constraints,' ICCAR, 2017.
- [4] Liang et al., 'An Energy Sharing Mechanism for a Freeform Robotic System - FreeBOT,' ICRA, 2022.
- [5] Wang et al., 'A Graph-Theory-Based Energy Routing Algorithm in Energy Local Area Network,' IEEE TII, 2017.